

Rapport de stage Spécialité : Informatique

Développement d'un middleware léger pour l'orchestration d'APIs
CBS avec supervision et documentation intégrée

Réalisé par : Abker Ranim

Entreprise : IntechGeeks



Encadrant : Wael Saidi

Période de stage : 01/07/2025 - 31/08/2025

Table des matières

Introduction générale	1
1 Contexte général	3
1.1 Organisme d'accueil	4
1.1.1 Présentation d'IntechGeeks	4
1.1.2 Valeurs fondamentales	4
1.1.3 Structure organisationnelle	5
1.2 Présentation du sujet	5
1.2.1 Contexte général du projet	5
1.3 Problématique	6
1.4 Étude de l'existant	6
1.4.1 Limites de la solution	7
1.5 Solution proposée	7
2 Analyse et Spécification des besoins	9
2.1 Spécification des acteurs	10
2.1.1 Identification des acteurs	10
2.2 Spécification des besoins	10
2.2.1 Besoins fonctionnels	10
2.2.2 Besoins non fonctionnels	10
2.2.3 Backlog Produit	10
2.3 Architecture du système	10
2.3.1 Architecture physique	10
2.3.2 Architecture logique	10

2.4	Technologies utilisées	10
3	Réalisation des tâches	11
3.1	Tâche 1 : Mise en route du CBS simulé via Docker	13
3.2	Tâche 2 : Développement du Middleware (APIs REST + appels CBS)	13
3.3	Tâche 3 : Intégration Swagger	13
3.4	Tâche 4 : Intégration OpenTelemetry	13
3.5	Tâche 5 : Développement du Dashboard React	13
	Conclusion générale et perspectives	14

Table des figures

1.1 Logo d’IntechGeeks 4

Liste des tableaux

Introduction générale

La gestion du crédit est un processus critique pour les établissements financiers, englobant l'entrée en relation client, la préparation des dossiers, l'évaluation des risques, la prise de décision et la traçabilité. Dans de nombreux environnements opérationnels, ces activités sont réparties entre des outils hétérogènes ou reposent sur des plateformes low-code qui imposent des contraintes en matière de scalabilité, d'interopérabilité et de maintenabilité à long terme.

Une analyse de DigiLAF, la solution de gestion du crédit existante déployée chez l'entreprise d'accueil, a révélé que si elle numérise avec succès le workflow central du crédit, son architecture introduit des limitations en matière de scalabilité, de flexibilité vis-à-vis des fournisseurs et d'auditabilité qui deviennent critiques à grande échelle. Ces contraintes motivent la conception d'une plateforme sur mesure construite sur des technologies ouvertes et maintenables.

Pour répondre à ces besoins, ce projet propose **CrediWise**, une plateforme web sécurisée organisée autour de domaines métier indépendants. Le rapport présente l'analyse, la conception et l'implémentation de la plateforme en dix chapitres, chaque chapitre-sprint correspondant soit à la fondation DevOps, soit à un ou plusieurs microservices de l'architecture cible alignés sur une phase du workflow de demande de crédit :

- **Chapitre 1 : Contexte du Projet** — présente l'entreprise d'accueil, le contexte du projet, la solution existante, la solution proposée et la méthodologie.
- **Chapitre 2 : Analyse et spécification des besoins** — Ce chapitre aborde la définition des exigences fonctionnelles et non fonctionnelles. Nous y présentons en détail la planification des sprints ainsi que la gestion du backlog produit, tout en décrivant l'architecture technique de l'application.
- **Chapitre 3 : Sprint 1, Authentification et gestion de la clientèle** — présente la base de sécurité fondée sur Keycloak, ainsi que l'administration des agences, des gestionnaires et des rôles. Il introduit également la gestion des dossiers clients, les

données de référence et l'exposition du domaine client aux autres services via les mécanismes d'interconnexion entre services.

- **Chapitre 4 : Sprint 2, Service de Demande de Crédit** — détaille la création des demandes, la gestion des brouillons, le routage basé sur les produits et les transitions d'état du cycle de vie.
- **Chapitre 5 : Sprint 3, Service d'Analyse** — décrit le dossier d'analyse, le workflow par étapes et le calcul du scoring pour les produits éligibles à l'analyse.
- **Chapitre 6 : Sprint 4, Service Checklist** — présente la checklist pré-comité qui consolide les documents et les sections de vérification requis avant la phase comité.
- **Chapitre 7 : Sprint 5, Service d'Analyse des Risques** — décrit le formulaire d'analyse du risque de crédit (CRA), les modules d'avis et de Q&R, ainsi que la piste d'audit associée.
- **Chapitre 8 : Sprint 6, Services du comité de crédit et de décaissement** — présente le procès-verbal de décision du comité ainsi que le procès-verbal de décaissement, et la clôture du cycle de vie de la demande de crédit.
- **Chapitre 9 : Sprint 0, Fondation DevOps et Observabilité** — établit le pipeline CI/CD Azure DevOps, l'automatisation de build Maven, la génération d'images Docker, la topologie docker-compose de production avec le reverse proxy Nginx, l'instrumentation OpenTelemetry de chaque service backend, et la stack d'observabilité SigNoz utilisée pour le tracing distribué, les métriques et les logs centralisés.
- **Chapitre 10 :** —

Contexte général

Plan

1	Organisme d'accueil	4
2	Présentation du sujet	5
3	Problématique	6
4	Étude de l'existant	6
5	Solution proposée	7

Introduction

Ce chapitre présente le contexte général du projet CrediWise. Il introduit l'entreprise d'accueil IntechGeeks, décrit le secteur de la microfinance en Tunisie, qui motive le projet, analyse les points forts et les limites de la solution existante DigiLAF, présente la plateforme CrediWise proposée et expose la méthodologie Scrum adoptée pour le développement.

1.1 Organisme d'accueil

1.1.1 Présentation d'IntechGeeks

IntechGeeks est une société d'ingénierie et d'intelligence artificielle au service de clients en Europe et en Afrique. Ses activités couvrent l'architecture logicielle cloud-native, l'intégration de systèmes et l'intelligence artificielle appliquée, avec un accent particulier sur la modernisation des systèmes d'information organisationnels. La figure ?? présente le logo de l'entreprise.



Figure 1.1 : Logo d'IntechGeeks

1.1.2 Valeurs fondamentales

Le travail de l'entreprise s'articule autour de principes qui guident la collaboration et la réalisation technique :

- **Rigueur technique** : Les systèmes livrés sont testés, documentés et instrumentés. Les tests automatisés, les pipelines reproductibles et l'observabilité sont considérés comme des critères essentiels de qualité du projet.

- **Immersion avant exécution** : Avant le début du développement, l'entreprise analyse le contexte opérationnel du client, les systèmes existants, les contraintes et les flux de travail, afin que la solution proposée corresponde aux besoins réels de l'entreprise.
- **Transfert de connaissances** : La livraison du projet comprend une documentation, l'explication des choix techniques et un accompagnement destiné à aider les équipes clientes à exploiter la solution livrée.
- **Transparence** : L'avancement du projet, les décisions techniques et les risques identifiés sont communiqués aux parties prenantes tout au long de la mission.

1.1.3 Structure organisationnelle

1.2 Présentation du sujet

Ce projet de fin d'études porte sur la conception et le développement de **CrediWise**, une plateforme web de gestion du crédit. L'objectif est de digitaliser et sécuriser l'ensemble du cycle de vie du crédit, de l'entrée en relation client jusqu'à la décision finale et la traçabilité.

La solution s'inscrit dans une architecture moderne basée sur des microservices et vise à réduire la dépendance aux solutions low-code existantes tout en améliorant la flexibilité, la scalabilité et la maintenabilité du système.

1.2.1 Contexte général du projet

Le secteur de la microfinance en Tunisie est supervisé par l'Autorité de Contrôle de la Microfinance (ACM). Les principaux acteurs — AKDI, Al Amal Microfinance, Zitouna Tamkeen, Advans Tunisie, Taysir Microfinance et Enda Tamweel, entre autres — jouent un rôle essentiel dans l'inclusion financière et le soutien des micro-entrepreneurs.

Malgré la croissance du secteur, la gestion du crédit reste un processus complexe, fortement documentaire et encore peu digitalisé en Tunisie et en Afrique en général.

Dans ce contexte, IntechGeeks a initié **CrediWise** comme module de gestion du crédit intégré à la plateforme KAYA, un middleware cloud-native permettant l'interconnexion de

systèmes financiers hétérogènes (Core Banking, CRM et ERP).

1.3 Problématique

Les solutions existantes de gestion du crédit souffrent de plusieurs limites, notamment en matière de scalabilité, de flexibilité et de maintenabilité.

Les processus restent souvent fragmentés, fortement dépendants de plateformes low-code propriétaires, ce qui entraîne :

- une limitation des performances sous forte charge,
- un coût élevé d'exploitation,
- une faible interopérabilité avec les systèmes externes,
- une traçabilité insuffisante des décisions,
- et une absence d'automatisation avancée (scoring, règles métier).

Ces contraintes rendent nécessaire la conception d'une solution moderne, modulaire et évolutive capable de supporter des processus de crédit complexes.

1.4 Étude de l'existant

La solution étudiée dans ce projet est **DigiLAF**, une application de gestion des crédits développée avec Microsoft Power Apps (Canvas App) et déployée dans plusieurs institutions financières en Tunisie et en Afrique.

Elle digitalise l'ensemble du cycle de crédit, de la création de la demande jusqu'au déblocage final, via un workflow basé sur le Business Process Flow.

Le processus comprend cinq étapes principales :

- **Analyse** : évaluation initiale et génération d'un rapport préliminaire.
- **Checklist** : vérification des documents requis.
- **Analyse des risques** : évaluation de la solvabilité et du risque.
- **Revue comité** : validation et décision du comité de crédit.

- **Visite de gestion** : contrôle terrain si nécessaire.

Power Automate est utilisé pour automatiser les transitions et les notifications, tandis que la gestion des accès repose sur les rôles de la Power Platform.

1.4.1 Limites de la solution

Malgré ses avantages, DigiLAF présente plusieurs limitations :

- scalabilité limitée,
- coût élevé (licences et connecteurs),
- dépendance forte à Microsoft,
- limitation de volume de données (2 000 enregistrements par requête),
- fragmentation du workflow,
- absence d'IA et de scoring automatique,
- faible observabilité et traçabilité,
- absence de portail client.

1.5 Solution proposée

Pour répondre à ces limites, ce projet propose **CrediWise**, une plateforme web de gestion du crédit basée sur une architecture modulaire et indépendante des solutions low-code.

La plateforme centralise la gestion des clients, des demandes de crédit, des analyses, des décisions et de la traçabilité dans une interface unifiée.

Fonctionnalités principales

- **Administration des utilisateurs et agences** : gestion des rôles et profils.
- **Gestion des clients** : centralisation des données clients.
- **Gestion des demandes de crédit** : création, brouillons, garanties et export PDF.
- **Routage par produit** : orientation automatique du workflow.
- **Analyse du crédit** : dossier structuré avec étapes d'évaluation.

- **Scoring et règles métier** : aide à la décision et traçabilité.
- **Modules de contrôle** : checklist, risques et comité de crédit.
- **Suivi du décaissement** : clôture du cycle de vie du crédit.

Sur le plan architectural, CrediWise repose sur une séparation claire entre frontend, backend, gestion d'identité (Keycloak) et bases de données. Cette architecture améliore la scalabilité, la maintenabilité et la traçabilité, tout en répondant aux limites de la solution existante.

Conclusion

Analyse et Spécification des besoins

Plan

1	Spécification des acteurs	10
2	Spécification des besoins	10
3	Architecture du système	10
4	Technologies utilisées	10

Introduction

2.1 Spécification des acteurs

2.1.1 Identification des acteurs

2.2 Spécification des besoins

2.2.1 Besoins fonctionnels

2.2.2 Besoins non fonctionnels

2.2.3 Backlog Produit

2.3 Architecture du système

2.3.1 Architecture physique

2.3.2 Architecture logique

2.4 Technologies utilisées

Conclusion

Réalisation des tâches

Plan

1	Tâche 1 : Mise en route du CBS simulé via Docker	13
2	Tâche 2 : Développement du Middleware (APIs REST + appels CBS)	13
3	Tâche 3 : Intégration Swagger	13
4	Tâche 4 : Intégration OpenTelemetry	13
5	Tâche 5 : Développement du Dashboard React	13

3.1 Tâche 1 : Mise en route du CBS simulé via Docker

Introduction

Plan de réalisation

Besoins / Cas d'utilisation

Conception

Réalisation

Conclusion

3.2 Tâche 2 : Développement du Middleware (APIs REST + appels CBS)

Introduction

Plan de réalisation

Besoins / Cas d'utilisation

Conception

Réalisation

Conclusion

3.3 Tâche 3 : Intégration Swagger

Introduction

Plan de réalisation

Besoins / Cas d'utilisation

Conclusion générale et perspectives